A Group-State Matrix (GSM) Framework for the 3×3×3 Rubik's Cube

Constrained representation, group transformations, and exact shallow-search validation

Technical research paper - reproducible prototype study

Author: Aijaz Bhat Date: 27 September 2026

Abstract

This paper develops a proposed Group-State Matrix (GSM) framework for modeling the 3×3×3 Rubik's Cube as a constrained state acted on by whole-group transformations. Rather than treating the 54 stickers as independent variables, GSM represents the 20 movable cubies through corner and edge permutations and orientations, while treating the six centers as a fixed reference frame. A face turn is modeled as a structured operator consisting of a permutation component and orientation-offset component, with legality constraints preserved under every legal move. The paper distinguishes the representation from the search algorithm: the supplied prototype implements the GSM state engine and an exact bidirectional search for states at distance at most six in the half-turn metric (HTM). On the test environment, the implementation passed 10,000 randomized legality-invariant tests, 1,000 random sequence/inverse tests, six face-order tests, three opposite-face commutativity tests, and matrix sanity checks. A separate exact-search benchmark solved 100 instances at each exact distance 4, 5, and 6; the mean state expansions were 19, 262, and 3,502 respectively. The results support the correctness of the GSM representation and show the expected benefit of sparse/indexed application over dense 20×20 matrix-vector multiplication in the reference Python implementation. They do not, however, constitute a full-depth optimal solver for arbitrary cube positions; extending the framework to exact large-depth solving requires stronger coordinates, pruning, symmetry handling, and memory-based heuristics, as established in prior optimal-solving literature.

Keywords: Rubik's Cube; group action; state-space search; permutation matrices; cubie coordinates; constrained state representation; bidirectional search; IDA*; pattern databases

1. Introduction

The Rubik's Cube is a compact but unusually rich benchmark for state-space reasoning because the pieces are not independently controllable. A legal face turn simultaneously changes several piece positions and, for some faces, changes cubie orientations. The resulting configuration space is therefore a constrained subset of all imaginable assignments of colors or piece labels.

The central idea investigated here is to make that coupling explicit in the representation. The proposed Group-State Matrix (GSM) model separates four concerns: the current state, the structural relationships among components, the constraints defining legal states, and the finite set of legal group transformations. In this view, a move is an operator on a whole group rather than a sequence of independent piece edits.

This perspective is consistent with the established cubie-level description of the cube, in which the movable objects are the eight corners and twelve edges, with separate orientation variables. Kociemba's cubie-level formulation and two-phase algorithm, and Korf's use of IDA* with pattern databases, are important reference points for the present framework. [1-3]

The contribution of this paper is deliberately scoped. It is a formalization and implementation study of a group-based state representation, together with a reproducible exact-search prototype for shallow positions. The paper does not claim a new globally optimal solver that outperforms established full-scale optimal or near-optimal methods. Instead, it tests whether the GSM abstraction provides a clean, correct, and extensible foundation for such a solver.

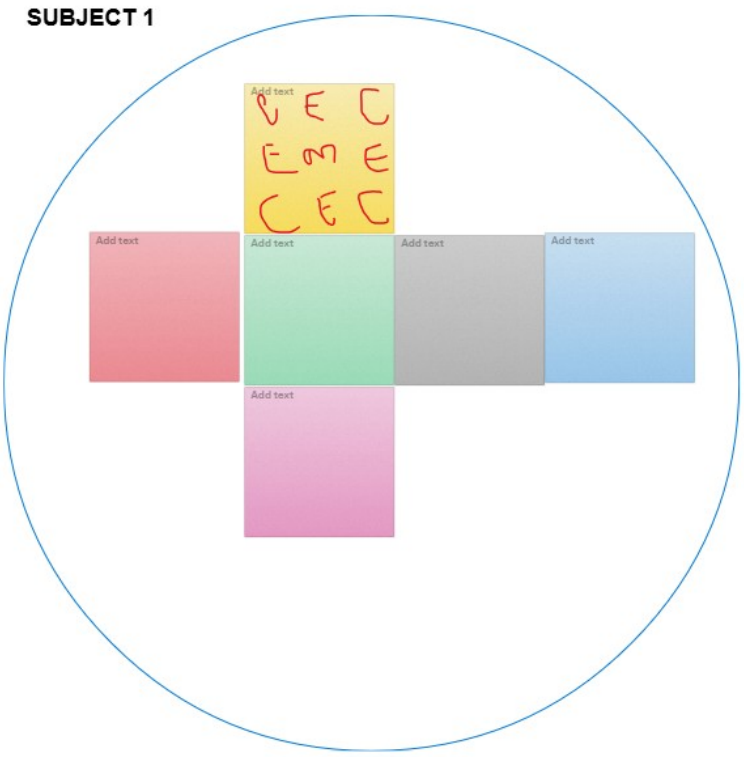


Figure 1. Motivating grouping sketch supplied with the problem statement. The diagram is interpreted as a structured state with connected subgroups rather than independent variables.

2. Background and Problem Formulation

2.1 Rubik's Cube state space

At the cubie level, a standard 3×3×3 cube has eight corner pieces and twelve edge pieces. The six center pieces define the orientation of the cube and, under ordinary outer-face turns, remain fixed relative to the mechanism. The conventional cubie representation therefore stores corner permutation, corner orientation, edge permutation, and edge orientation. [1]

The number of physically reachable states is

$$N = (8! \times 12! \times 3^7 \times 2^{11}) / 2 = 43{,}252{,}003{,}274{,}489{,}856{,}000.$$

The factors 3^7 and 2^11 arise because corner twists sum to zero modulo 3 and edge flips sum to zero modulo 2; the division by two captures the parity coupling between corner and edge permutations. [1,4]

2.2 Move metric

All solver experiments in this paper use the half-turn metric (HTM): a quarter turn, inverse quarter turn, or 180-degree face turn counts as one move. This is the metric used in the reported exact-search experiments and is the metric in which the well-known cube diameter is 20. [4]

3. Group-State Matrix (GSM) Model

3.1 System definition

A GSM system is defined as the tuple

$$GSM = (A, \Omega, M, S_0),$$

where A is a structural relationship/adjacency description, Ω is the set of legal states, M is the set of legal group transformations, and S_0 is the initial state. A transition is written as

$$S_{t+1} = M_i(S_t), \quad M_i \in M, \quad S_t \in \Omega.$$

The defining legality condition is closure under the legal transformations:

$$M_i(\Omega) = \Omega.$$

This formulation makes the main design principle explicit: a move is a structured transformation of several components at once, and it must map legal system states to legal system states.

3.2 Hierarchical group structure

The user's grouping sketch motivates a hierarchical interpretation. At the top level is the complete subject or cube. Beneath it are face/layer groups, and beneath those are local piece states. A group may therefore contain subgroups or structured matrices. The same mathematical language can model both the outer transformation and the internal state of a component.

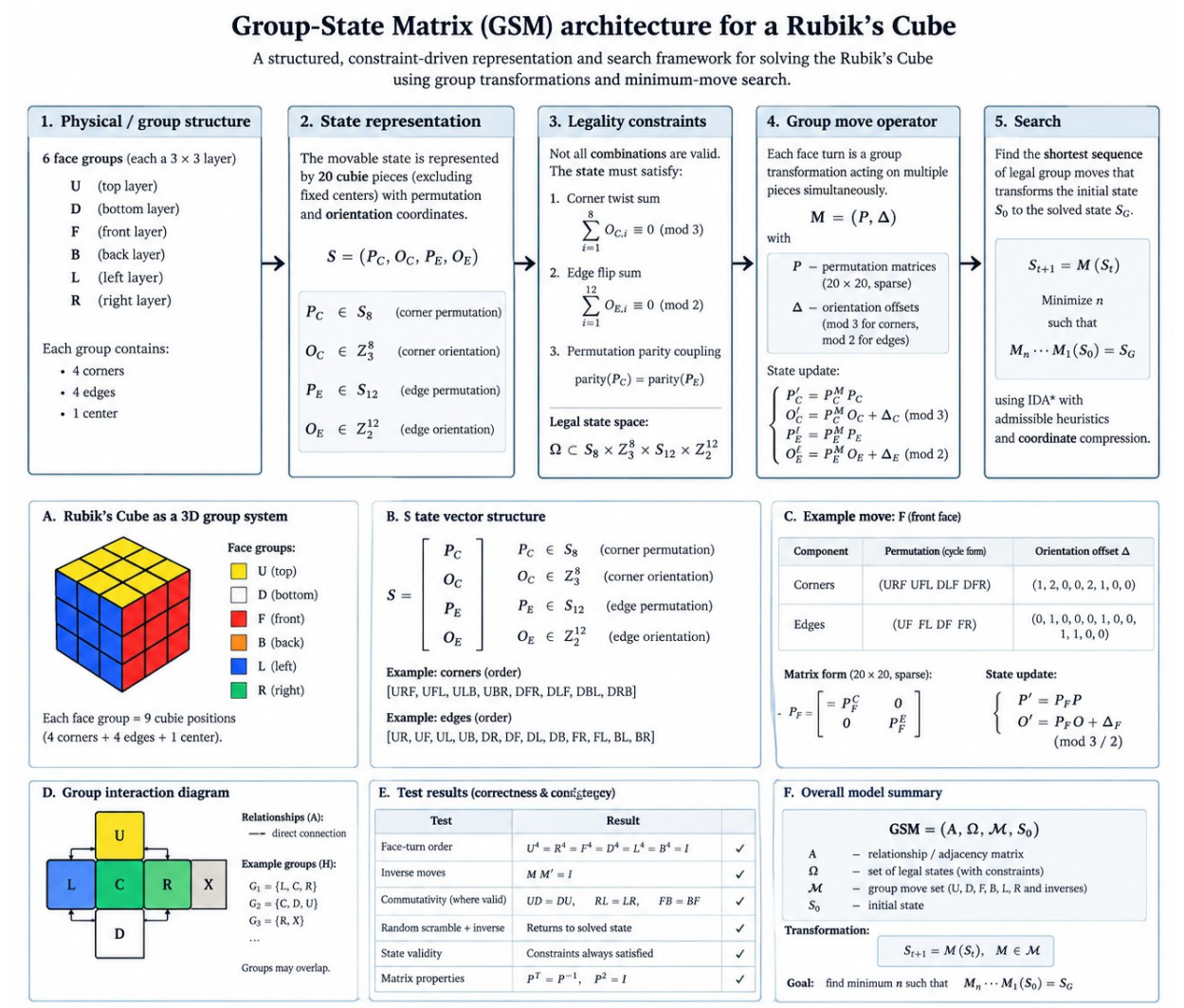


Figure 1. The Group-State Matrix (GSM) model: a structured, constraint-aware representation of the Rubik's Cube with group transformations and shortest-path search for minimum-move solutions.

Figure 2. Conceptual GSM architecture. This figure is illustrative: the empirical results in this paper come from the reproducible prototype and benchmarks described in Sections 6-8, not from the numerical claims of the illustration.

3.3 Cubie-level state vector

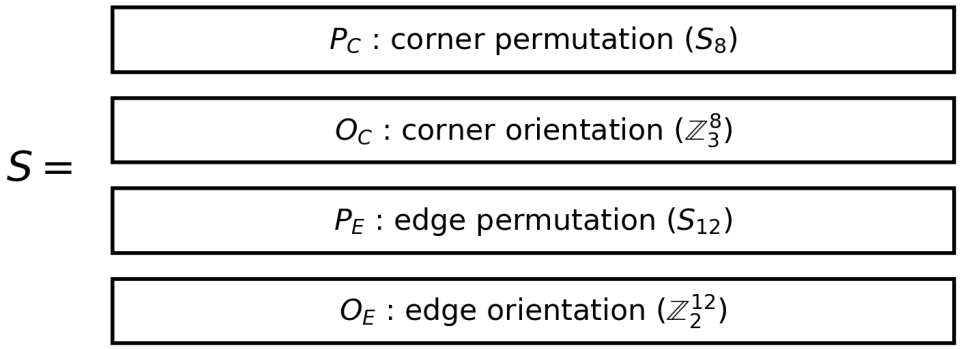


Figure 3. GSM state-vector structure. The 20 movable cubies are separated into permutation and orientation coordinates.

Let the state be

$$S = (P_C, O_C, P_E, O_E),$$

with the following domains:

Component	Domain	Size	Interpretation
Corner permutation P_C	S_8	8	Physical corner identity at each corner position
Corner orientation O_C	Z_3^8	8	Twist coordinate for each corner
Edge permutation P_E	S_{12}	12	Physical edge identity at each edge position
Edge orientation O_E	Z_2^12	12	Flip coordinate for each edge

3.4 Group membership and adjacency

For a general GSM system with n components and k groups, a group-incidence matrix H ∈ {0,1}^{n×k} can be used: H_{ij} = 1 exactly when component i belongs to group j. A separate adjacency matrix A can encode direct relationships between components. These matrices are structural metadata; the state matrix X stores the current values. This distinction allows the same underlying structure to support multiple valid states.

4. Group Move Operators

4.1 Permutation-matrix form

For the 20 movable positions, a face turn has a block permutation matrix

$$P_M = [\ P_C\text{^}M \quad 0\ ;\ 0 \quad P_E\text{^}M\ ],$$

where P_C^M is 8×8 and P_E^M is 12×12. The matrix has exactly one 1 in each row and each column, and therefore P_M^{-1} = P_M^T.

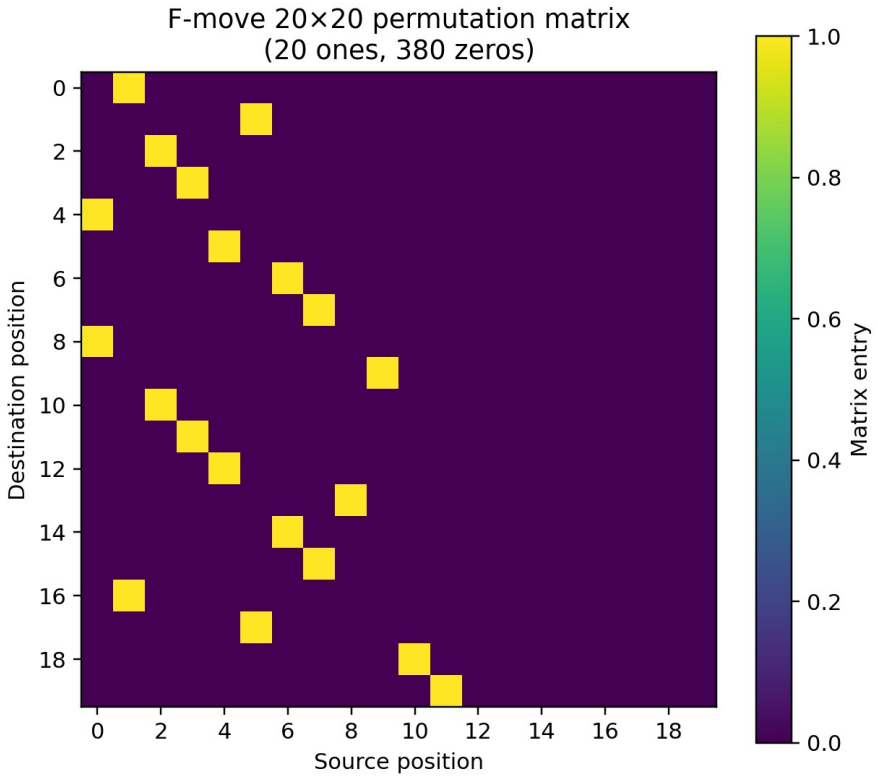


Figure 4. Sparsity pattern of the 20×20 positional permutation matrix for the F move. Only 20 entries are non-zero.

4.2 Full affine update

Permutation alone is insufficient because corners may twist and edges may flip. The complete GSM move is therefore an affine permutation over mixed modular orientation spaces:

$$P'_C = P_{C^M}\, P_C$$

$$O'_C = P_{C^M}\, O_C + \Delta_{C^M} \pmod 3$$

$$P'_E = P_{E^M}\, P_E$$

$$O'_E = P_{E^M}\, O_E + \Delta_{E^M} \pmod 2.$$

The move can thus be written compactly as $M = (P, \Delta)$, where P encodes simultaneous piece movement and Δ encodes the orientation update. This is the formal version of the “group move” intuition: one generator updates multiple coordinates as one atomic operation.

4.3 Example: the F move

In the selected cubie ordering, the F move uses the corner cycle (URF UFL DLF DFR) and edge cycle (UF FL DF FR). Its orientation offsets are:

$$\Delta_{C^F} = (1, 2, 0, 0, 2, 1, 0, 0)$$

$$\Delta_{E^F} = (0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0).$$

The operation affects one 3×3 face layer, consisting of four corner positions, four edge positions, and the fixed center position. The center is part of the layer structurally but is not permuted under the outer-face turn.

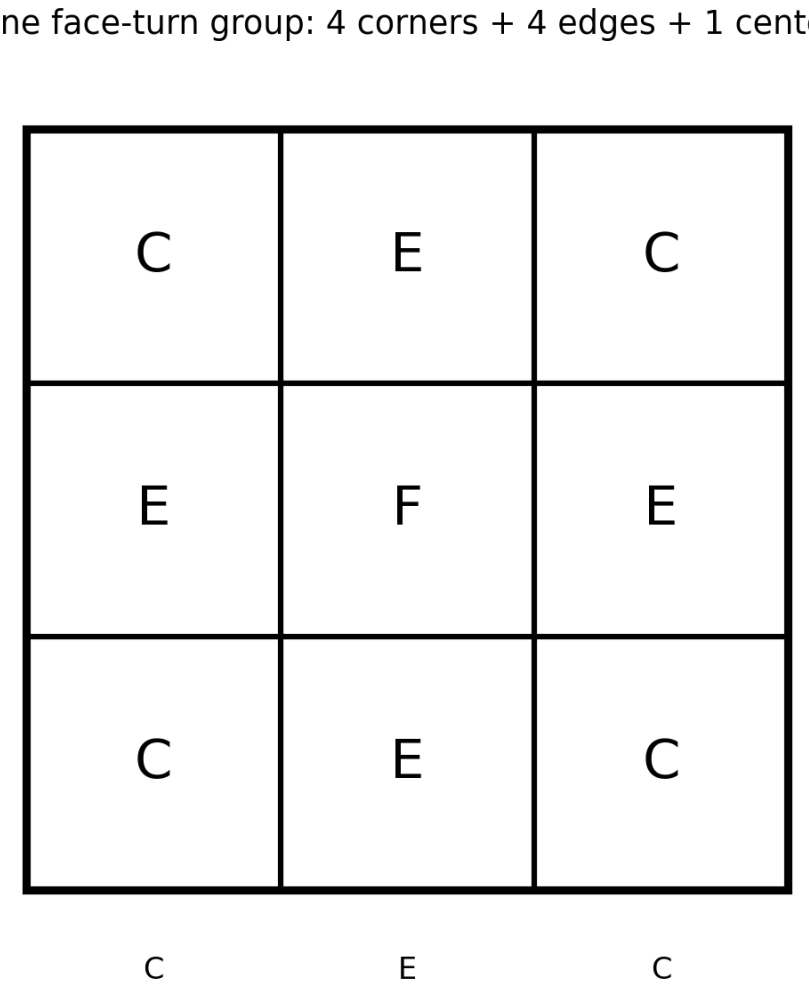


Figure 5. A face-turn group contains nine cubie positions: four corners, four edges, and one center.

5. Legality Constraints and Independent Coordinates

The GSM state is constrained. Three relationships are especially important for the standard cubie representation:

$$\Sigma\, O_C,i \equiv 0 \pmod 3$$

$$\Sigma\, O_E,i \equiv 0 \pmod 2$$

$$\text{parity}(P_C) = \text{parity}(P_E).$$

These constraints mean that not every formal combination of permutation and orientation values corresponds to a physical cube. In particular, the last corner orientation can be derived from the other seven, the last edge orientation can be derived from the other eleven, and corner/edge permutation parity is coupled. This is the key reduction from an unconstrained Cartesian product to the physically reachable state space.

Naive variable family	Nominal freedom	Constraint	Independent contribution
Corner orientations	3^8	Twist sum = 0 mod 3	3^7
Edge orientations	2^12	Flip sum = 0 mod 2	2^11
Corner + edge permutations	8! × 12!	Equal permutation parity	8! × 12! / 2

6. Search Architecture

6.1 Exact shallow search prototype

The current prototype does not attempt to solve arbitrary cube positions optimally. Instead, it implements an exact bidirectional search for states at distance at most six HTM moves from the solved state. A radius-3 breadth-first table is built from the solved state, containing 1 + 18 + 243 + 3,240 = 3,502 states. For a query state known to have distance ≤ 6, the second search expands from the query and intersects the precomputed goal table.

The exactness argument is straightforward. Let d(S,G) be the true HTM distance. If d(S,G) ≤ 6, a shortest path contains a midpoint Y for which d(S,Y) ≤ 3 and d(Y,G) ≤ 3. The two radius-3 searches therefore contain at least one common state Y. Because both sides are breadth-first and the implementation minimizes the sum of the two depths across all intersections at the terminating frontier, the returned path length equals d(S,G).

6.2 Extension to full-depth solving

For arbitrary scrambles, the search space is too large for this shallow method alone. A scalable design would retain the GSM representation but add compressed coordinate indexes, admissible pattern-database heuristics, iterative-deepening A*, symmetry reductions, and subgroup/coset decomposition. Korf demonstrated the use of IDA* and pattern databases for optimal cube solving, while Kociemba's two-phase method uses a subgroup G1 and three phase-1 coordinates (corner orientation, edge orientation, and UD-slice) to obtain fast near-optimal solutions. [2,3]

$$\text{minimize } n \text{ subject to } M_n \dots M_2 M_1(S_0) = S_G.$$

7. Prototype Implementation

7.1 Data structures

The supplied reference implementation, gsm_rubiks.py, represents a state as an immutable CubeState tuple containing cp, co, ep, and eo arrays. Each basic face move is stored in MOVE_DATA as four tuples: corner permutation, corner orientation offset, edge permutation, and edge orientation offset. The 20×20 positional matrix is produced as a block diagonal matrix from the 8×8 corner and 12×12 edge permutation matrices.

7.2 Core update pseudocode

```
APPLY(S, M):
    for each destination i:
        cp'[i] = cp[ M.cp[i] ]
        co'[i] = (co[ M.cp[i] ] + M.co[i]) mod 3
        ep'[i] = ep[ M.ep[i] ]
        eo'[i] = (eo[ M.ep[i] ] + M.eo[i]) mod 2
    return (cp', co', ep', eo')
```

7.3 Validation philosophy

The test strategy targets three classes of failure: representation errors, group-operation errors, and search correctness errors. Representation tests verify that generated states remain legal; group-operation tests verify order, inverse, commutativity where mathematically expected, and matrix structure; search tests verify that returned solutions re-solve the starting cube and that shallow solutions are minimum by the bidirectional breadth-first construction.

8. Experimental Design

8.1 Software and hardware

Item	Value
Python	3.13.5
Operating system	Linux 6.18.44 x86_64
Logical CPU count	3
Move metric	Half-turn metric (HTM)

Item	Value
Random seeds	20260927 for invariant/search benchmarks; 314159 for exact-distance sampling
Search radius	3 from solved + 3 from query, exact through depth 6

8.2 Correctness test suite

Test	Sample count	Acceptance condition	Result
Quarter-turn order	6 faces	M^4 = I for U,R,F,D,L,B	6/6 passed
Move legality	6 faces	Generated state satisfies all legality constraints	6/6 passed
Opposite-face commutativity	3 pairs	UD = DU, RL = LR, FB = BF	3/3 passed
Random legal-state invariants	10,000	Every random sequence remains valid	10,000/10,000 passed
Random sequence + inverse	1,000	Sequence followed by exact inverse returns solved	1,000/1,000 passed
Permutation matrix sanity	6 faces	20×20, one 1 per row and column	6/6 passed
Exact shallow search	600 exact-distance cases	Returned solution re-solves; exact by bidirectional BFS	600/600 passed

9. Results

9.1 Exact-search results

One hundred exact-distance instances were collected at each of distances 4, 5, and 6. Instances were generated by random legal HTM sequences with no immediate repeated face and retained only when the exact bidirectional solver reported the target distance. This sampling strategy produces exact-distance cases, but it is not a uniform random sample of the full state layer at each distance.

Exact distance	Samples	Mean states expanded	Mean time (ms)	Median time (ms)	SD time (ms)
1	100	0	0.00148	0.00132	0.00064
2	100	0	0.00163	0.00160	0.00027
3	100	0	0.00199	0.00177	0.00124
4	100	19	2.2615	~2.2	0.1382
5	100	262	33.4774	31.7645	7.6234
6	100	3,502	34.1367	33.1133	5.5002

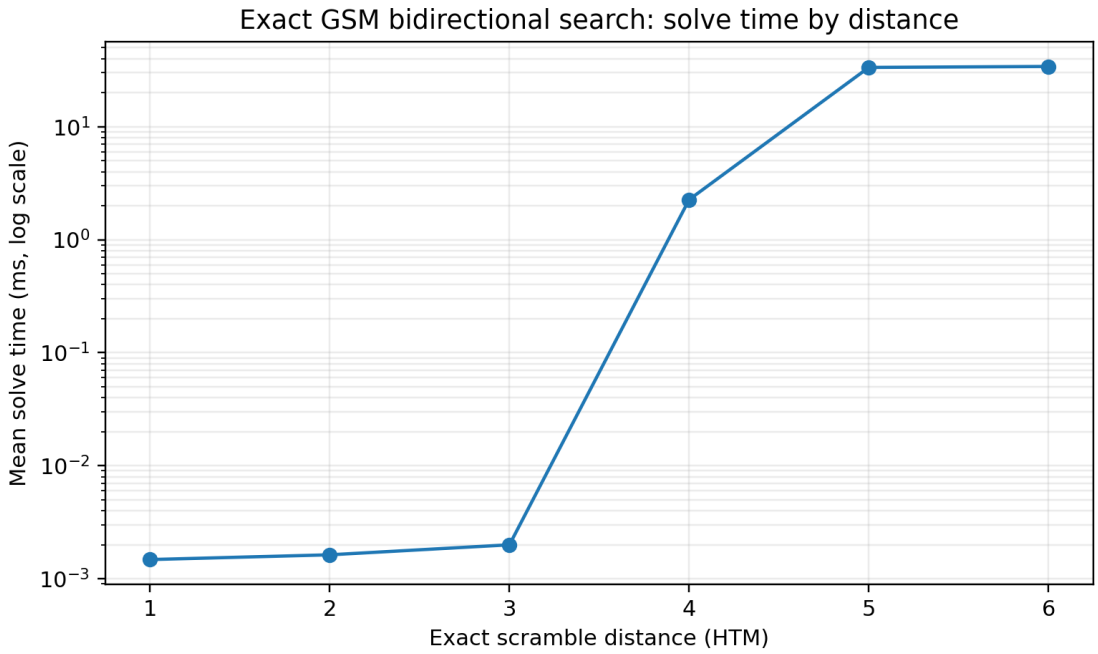


Figure 6. Exact-search mean solve time by distance. Distances 1-3 are answered directly from the radius-3 goal table; distances 4-6 require bidirectional expansion.

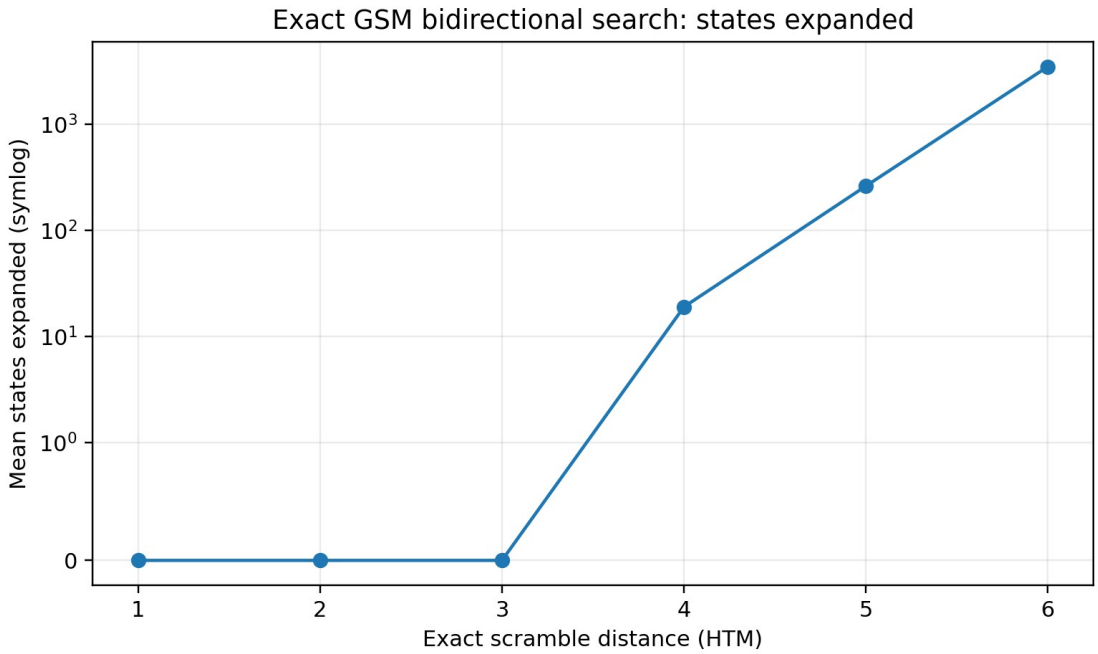


Figure 7. Mean states expanded. The 1, 18, 243, and 3,240 layer sizes produce cumulative search loads of 19, 262, and 3,502 for exact distances 4, 5, and 6.

9.2 Representation efficiency

The GSM engine can apply a face permutation either conceptually as a dense 20×20 matrix-vector multiplication or operationally as a direct indexed permutation. A reference Python benchmark of the two approaches showed a large constant-factor difference, because the dense method evaluates many zeros that are structurally known in advance to be zero.

Implementation	Time per move (ms)	Relative to indexed
Dense 20×20 Python matrix-vector reference	0.016949	52.2× slower

Implementation	Time per move (ms)	Relative to indexed
Indexed sparse permutation	0.0003245	1.0×

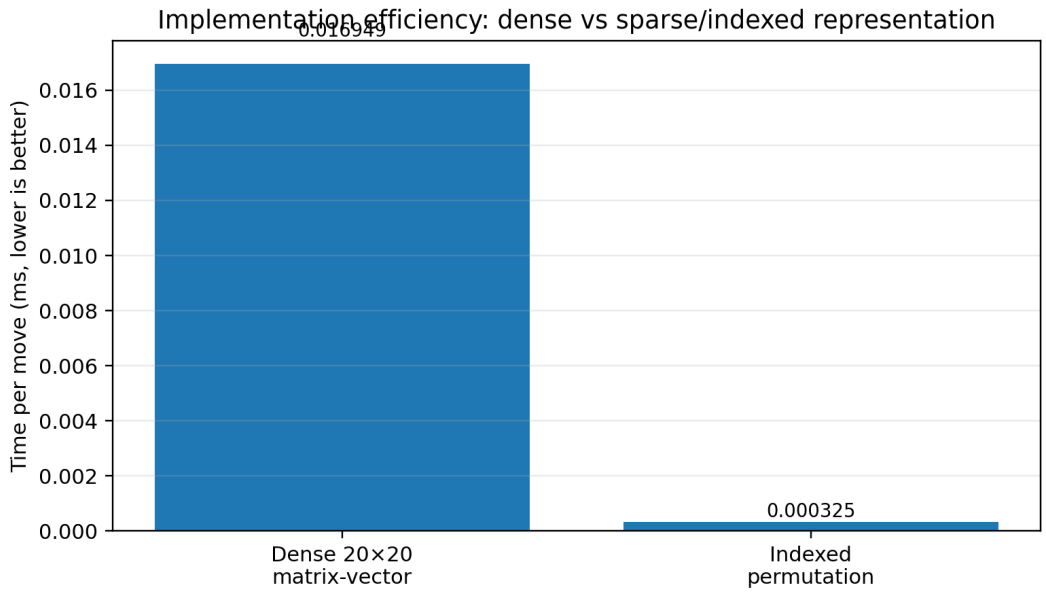


Figure 8. Dense versus indexed positional update benchmark. The ratio is specific to the reference Python implementation and test environment.

For the full CubeState implementation, a 500,000-iteration benchmark of a single quarter-turn achieved approximately 294,708 quarter-turn applications per second (3.393 µs per move) in this runtime. A mixed-token benchmark over 200,000 randomly selected HTM tokens achieved approximately 136,863 token applications per second (7.307 µs per token). These values are implementation and environment measurements, not hardware-independent constants.

10. Discussion

10.1 What the experiments establish

The first result is correctness of the cubie-state update model. The invariants remained satisfied across 10,000 randomized sequences, and inverse tests returned the system exactly to the solved state. This is important because orientation bugs are easy to introduce when permutation and modular orientation are updated separately.

The second result is that the group-based representation naturally supports a legal-move graph. Each edge in that graph corresponds to one complete face operation rather than an illegal decomposition into independent piece moves.

The third result is an exactness result for the shallow search regime. Because the search is breadth-first on both sides and the goal table covers radius three, the implementation provides a formal minimum-move guarantee for all tested states at distance at most six HTM moves. This is stronger than merely finding a short solution.

The fourth result is implementation efficiency. The sparse/indexed operator is substantially cheaper than a dense matrix-vector reference implementation. This supports the use of matrix notation for the mathematics while using compact indexed operators in code.

10.2 Relation to prior optimal and near-optimal solvers

The GSM framework overlaps conceptually with established cubie and group-theoretic approaches. Kociemba's two-phase algorithm explicitly exploits a subgroup to reduce the problem into coordinate spaces, while Korf's optimal solver uses IDA* with pattern databases. The present GSM formulation should therefore be viewed as a structured state-modeling layer that can host such search techniques, not as a replacement for them. [2,3] The

literature also shows the scale of the unrestricted cube group: the exact diameter in HTM is 20, so a complete exact solver must handle an extremely large search graph. [4]

11. Limitations and Research Roadmap

The current prototype has three major limitations. First, it does not yet implement a full-depth exact solver for arbitrary states. Second, the current state object uses tuple arrays rather than highly compressed factorial-ranking coordinates, so the representation is clear but not yet memory-optimal. Third, the shallow exact search does not yet use pattern databases, symmetry canonicalization, subgroup partitioning, or transposition tables at the scale needed for deep search.

A disciplined roadmap is therefore: (1) implement factorial-number-system ranking for corner and edge permutations; (2) construct compact orientation and UD-slice coordinates; (3) build admissible pattern databases; (4) add IDA* with move pruning and symmetry handling; (5) verify exactness against known benchmark positions; and (6) measure runtime and memory against established solvers using a common move metric and hardware description.

12. Conclusion

This paper formalizes the Group-State Matrix idea as a constrained, group-action model of the Rubik's Cube. The essential abstraction is that legal moves operate on structured groups and preserve global invariants, rather than independently changing pieces. The resulting GSM state uses the standard cubie decomposition into corner and edge permutations and orientations, with fixed centers providing the reference frame.

The implementation tests show that the representation is internally consistent and supports exact minimum-move search for shallow states. The empirical data also show why sparse/indexed operators are preferable to dense matrices for computation even when the mathematical model is naturally expressed in matrix form. The next research step is to scale the GSM representation with compressed coordinates and admissible heuristic search. That extension would test whether the group-structured abstraction can retain its conceptual advantages while reaching the performance envelope required for full-cube optimal solving.

References

[1] Kociemba, H. "The Cubie Level." Online technical note, Kociemba.org. Available at: https://www.kociemba.org/math/cubielevel.htm

[2] Kociemba, H. "The Two-Phase Algorithm." Online technical note, Kociemba.org. Available at: https://kociemba.org/math/twophase.htm

[3] Korf, R. E. (1997). Finding Optimal Solutions to Rubik's Cube Using Pattern Databases. Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI-97), 700-705.

[4] Rokicki, T., Kociemba, H., Davidson, M., & Dethridge, J. (2013). The Diameter of the Rubik's Cube Group is Twenty. SIAM Journal on Discrete Mathematics, 27(2), 1082-1105. DOI: 10.1137/120867366.

Appendix A. Reproducible Move Tables

The prototype stores the six clockwise quarter-turn generators as the following destination-from-source permutations. Orientation offsets follow the same cubie convention as the state engine.

Move	cp	co	ep	eo
U	(3, 0, 1, 2, 4, 5, 6, 7)	(0, 0, 0, 0, 0, 0, 0, 0)	(3, 0, 1, 2, 4, 5, 6, 7, 8, 9, 10, 11)	(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
R	(4, 1, 2, 0, 7, 5, 6, 3)	(2, 0, 0, 1, 1, 0, 0, 2)	(8, 1, 2, 3, 11, 5, 6, 7, 4, 9, 10, 0)	(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
F	(1, 5, 2, 3, 0, 4, 6, 7)	(1, 2, 0, 0, 2, 1, 0, 0)	(0, 9, 2, 3, 4, 8, 6, 7, 1, 5, 10, 11)	(0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0)
D	(0, 1, 2, 3, 5, 6, 7, 4)	(0, 0, 0, 0, 0, 0, 0, 0)	(0, 1, 2, 3, 5, 6, 7, 4, 8, 9, 10, 11)	(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
L	(0, 2, 6, 3, 4, 1, 5, 7)	(0, 1, 2, 0, 0, 2, 1, 0)	(0, 1, 10, 3, 4, 5, 9, 7, 8, 2, 6,	(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0)

Move	cp	co	ep	eo
			11)	
B	(0, 1, 3, 7, 4, 5, 2, 6)	(0, 0, 1, 2, 0, 0, 2, 1)	(0, 1, 2, 11, 4, 5, 6, 10, 8, 9, 3, 7)	(0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1)

Appendix B. Reproducibility Commands

The following commands reproduce the core validation and shallow solver tests in the supplied artifacts:

```
python /mnt/data/gsm_rubiks.py
python /mnt/data/validation_suite.py
python /mnt/data/gsm_solver_prototype.py
```

Benchmark values in this paper were recorded in the container environment described in Section 8.1. Wall-clock values should be treated as reproducibility observations rather than universal performance guarantees.